

by the great Jorge Luis Borges. A 1-gram is an individual word, so the 1-grams of this phrase are *reality*, *is*, *not*, *always*, *probable*, *or*, and *likely*. The 2-grams are every string of two words that appear together, including *realityis*, *is not*, *not always*, *always probable*, and so on. The 3-grams are *reality is not*, *is not always*, and so on.

Tokenizing and Getting N-grams

We'll use a Python module called `nltk` to make n-gram collection easy. We'll first tokenize our text. *Tokenizing* simply means splitting a string into its component words, ignoring punctuation. For example:

```
from nltk.tokenize import sent_tokenize, word_tokenize
text = "Time forks perpetually toward innumerable futures"
print(word_tokenize(text))
```

The result we see is this:

```
['Time', 'forks', 'perpetually', 'toward', 'innumerable', 'futures']
```

We can tokenize and get the n-grams from our text as follows:

```
import nltk
from nltk.util import ngrams
token = nltk.word_tokenize(text)
bigrams = ngrams(token, 2)
trigrams = ngrams(token, 3)
fourgrams = ngrams(token, 4)
fivegrams = ngrams(token, 5)
```

Alternatively, we can put all the n-grams in a list called `grams`:

```
grams = [ngrams(token, 2), ngrams(token, 3), ngrams(token, 4), ngrams(token, 5)]
```

In this case, we have gotten a tokenization and a list of n-grams for a short one-sentence text. However, in order to have an all-purpose phrase completion tool, we'll need a considerably larger corpus. The `brown` corpus we used for space insertion won't work because it consists of single words and so we can't get its n-grams.

One corpus we could use is a collection of literary texts made available online by Google's Peter Norvig at <http://norvig.com/big.txt>. For the